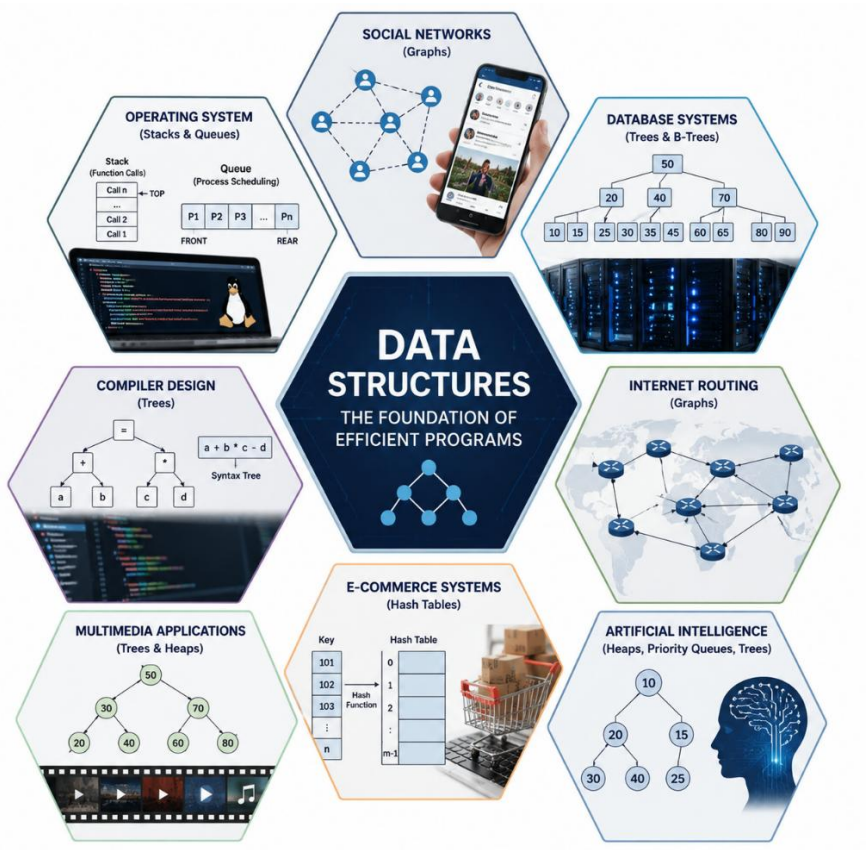
Data Structures & Algorithms Lab

ECE2132



Department of Electronics & Communication Engineering

By:

Dr. D.V. Prashant
Dr. Kamal Kishore Upadhyay

Manipal University Jaipur,

Dehmi Kalan, Off Jaipur-Ajmer Expressway, Jaipur, (Raj.)

Rajasthan 303007

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)

(AY 2026-27)

EXPERIMENT NO. 3

WRITE A PROGRAM IN C++ TO IMPLEMENT INSERTION OF AN ELEMENT IN AN ARRAY AT THE BEGINNING, ANY POSITION, AND THE END

AIM: To study and implement insertion of an element in an array at the beginning, at any specified position, and at the end using C++.

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: An array is a collection of elements stored in contiguous memory locations. Insertion is the process of adding a new element into an existing array. Since arrays have fixed positions, elements must be shifted to create space before inserting a new element.

- **Insertion at the Beginning:** All existing elements are shifted one position to the right.
- **Insertion at Any Position:** Elements from the specified position onward are shifted one position to the right.
- **Insertion at the End:** The new element is placed after the last existing element if space is available.

Array insertion is widely used in database management systems, inventory management, student record systems, and many other applications.

ALOGRITHM:

1. Start.
2. Declare an integer array.
3. Enter the number of elements.
4. Read the array elements.
5. Display the insertion menu.
6. If beginning is selected, shift all elements to the right and insert the new element at index 0.
7. If any position is selected, shift elements from that position to the right and insert the new element.
8. If end is selected, insert the new element after the last element.
9. Display the updated array.

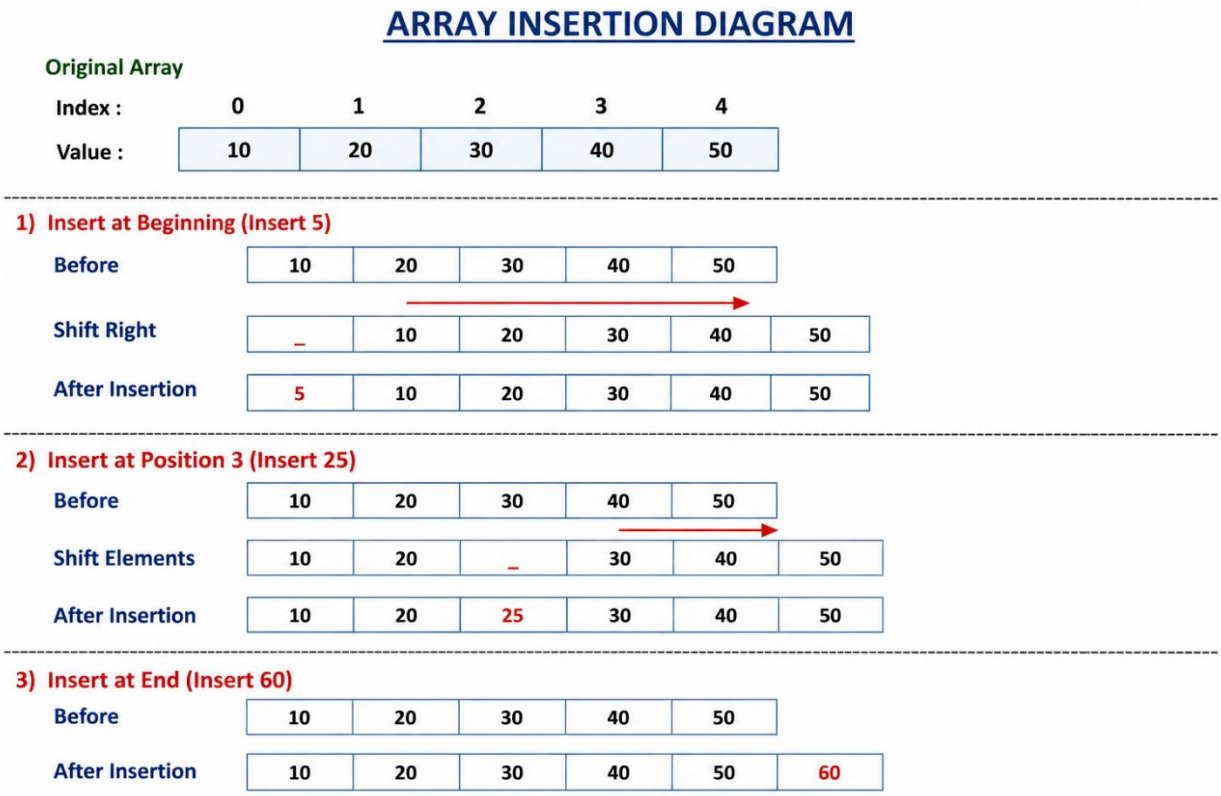


Figure.1: Insertion of Element in Array.

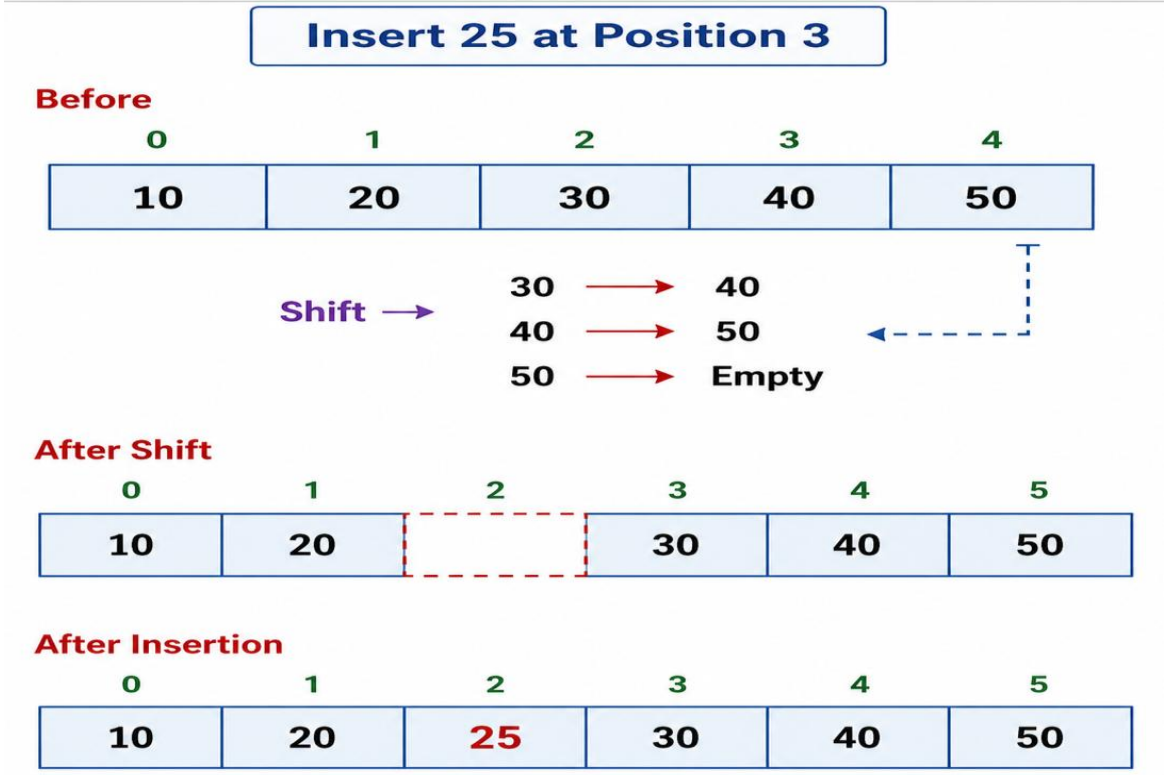


Figure.2: Shifting of Elements during Insertion.

PROGRAM (C++):

```
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout<<"Enter number of elements: ";
    cin>>n;

    cout<<"Enter array elements:\n";
    for(int i=0;i<n;i++)
        cin>>arr[i];

    cout<<"\n1. Insert at Beginning";
    cout<<"\n2. Insert at Any Position";
    cout<<"\n3. Insert at End";
    cout<<"\nEnter your choice: ";
    cin>>choice;

    if(choice==1)
    {
        cout<<"Enter value: ";
        cin>>value;

        for(int i=n;i>0;i--)
            arr[i]=arr[i-1];

        arr[0]=value;
        n++;
    }
    else if(choice==2)
    {
        cout<<"Enter Position: ";
        cin>>pos;

        cout<<"Enter Value: ";
        cin>>value;

        for(int i=n;i>=pos;i--)
            arr[i]=arr[i-1];

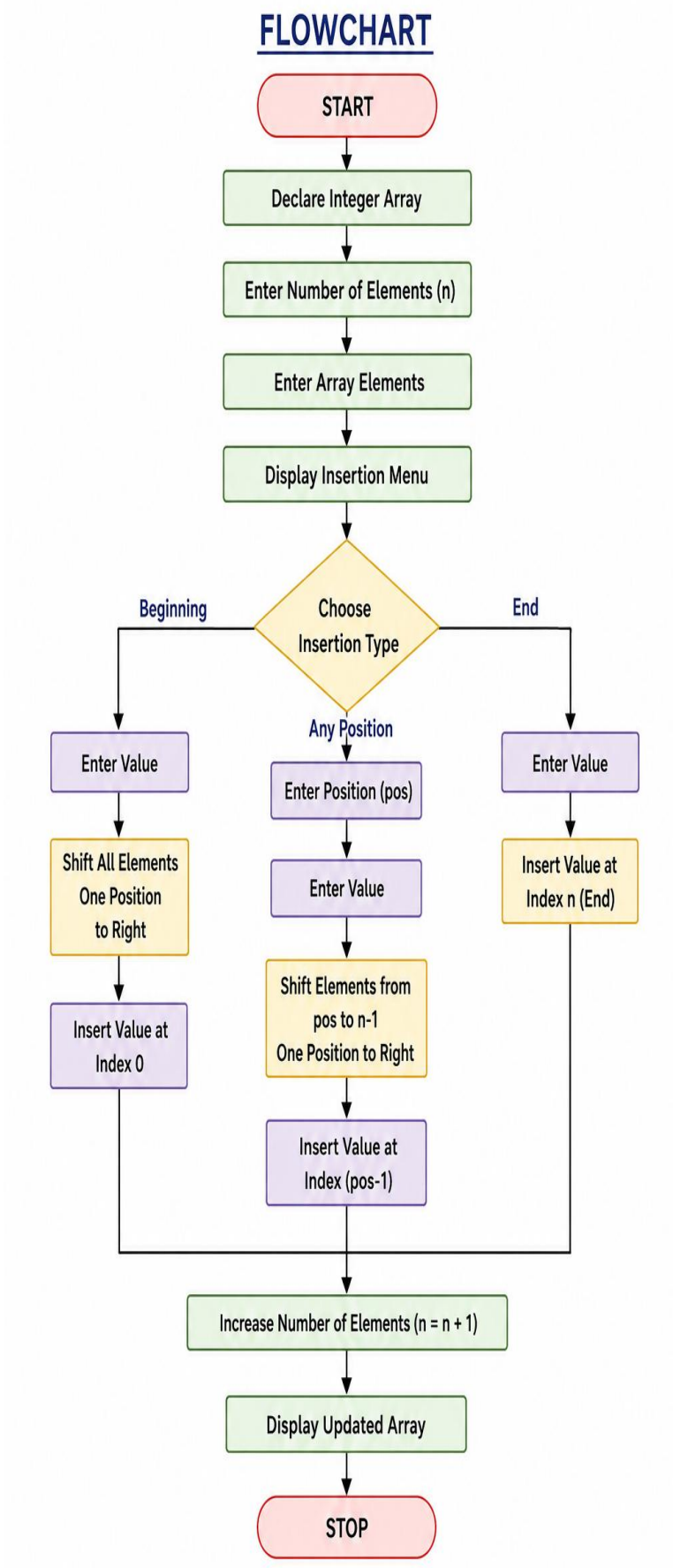
        arr[pos-1]=value;
        n++;
    }
    else if(choice==3)
    {
        cout<<"Enter Value: ";
        cin>>value;

        arr[n]=value;
        n++;
    }

    cout<<"\nUpdated Array:\n";

    for(int i=0;i<n;i++)
        cout<<arr[i]<<" ";

    return 0;
}
```



EXPLANATION OF THE PROGRAM:

Statement	Explanation
#include <iostream>	Provides input/output functions.
int arr[100];	Declares an integer array with extra space.
cin>>n;	Reads the number of elements in the array.
for(int i=0;i<n;i++)	Reads the array elements.
choice	Selects the type of insertion.
for(int i=n;i>0;i--)	Shifts all elements one position to the right for insertion at the beginning.
for(int i=n;i>=pos;i--)	Shifts elements to create space at the specified position.
arr[0]=value;	Inserts the element at the beginning.
arr[pos-1]=value;	Inserts the element at the specified position.
arr[n]=value;	Inserts the element at the end of the array.
cout	Displays the updated array.

Sample Input 1 (Beginning):
Enter number of elements: 5
10 20 30 40 50
Choice: 1
Sample Output:
Updated Array:
5 10 20 30 40 50
Sample Input 2 (Any Position):
Enter number of elements: 5
10 20 30 40 50
Choice: 2
Enter Position: 3
Enter Value: 25
Sample Output:
Updated Array:
10 20 25 30 40 50
Sample Input 3 (End):
Enter number of elements: 5
10 20 30 40 50

Choice: 3
Enter Value: 60
Sample Output:
Updated Array:
10 20 30 40 50 60
Time Complexity
Insertion at Beginning: **O(n)**
Insertion at Any Position: **O(n)**
Insertion at End: **O(1)** (when free space is available)
Space Complexity
O (1)
IMPLEMENTATION OF CODE:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout<<"Enter number of elements: ";
    cin>>n;

    cout<<"Enter array elements:\n";
    for(int i=0;i<n;i++)
        cin>>arr[i];

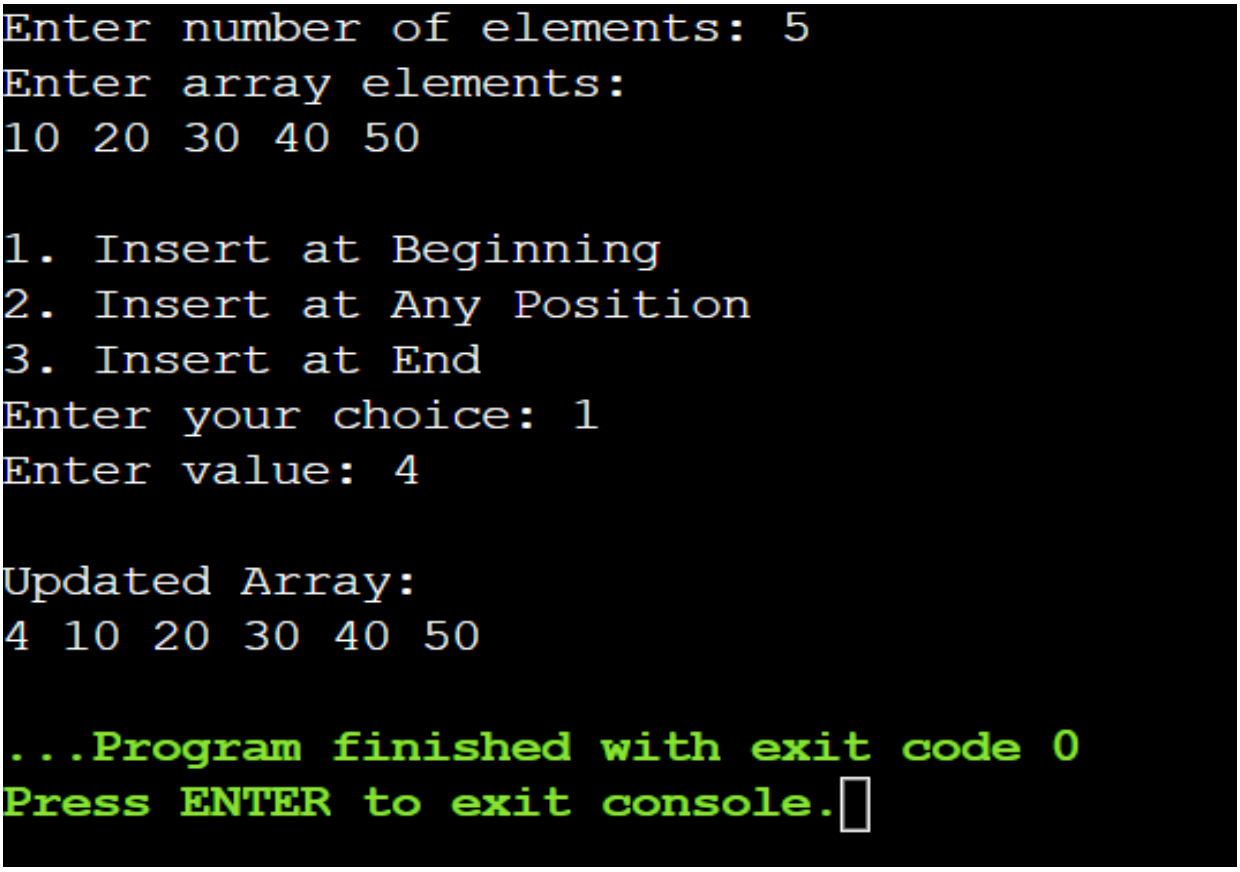
    cout<<"\n1. Insert at Beginning";
    cout<<"\n2. Insert at Any Position";
    cout<<"\n3. Insert at End";
    cout<<"\nEnter your choice: ";
    cin>>choice;

    if(choice==1)
    {
        cout<<"Enter value: ";
        cin>>value;
        for(int i=n;i>0;i--)
            arr[i]=arr[i-1];

        arr[0]=value;
        n++;
    }
    else if(choice==2)
    {
        cout<<"Enter Position: ";
        cin>>pos;

        cout<<"Enter Value: ";
        cin>>value;
        for(int i=n;i>=pos;i--)
            arr[i]=arr[i-1];

        arr[pos-1]=value;
        n++;
    }
    else if(choice==3)
    {
        cout<<"Enter Value: ";
        cin>>value;
        arr[n]=value;
        n++;
    }
    cout<<"\nUpdated Array:\n";
    for(int i=0;i<n;i++)
        cout<<arr[i]<<" ";
    return 0;
}
```



APPLICATIONS: Student Record Management, Inventory Management Systems, Database Management, Employee Record Systems, Dynamic Data Processing

RESULT: The program was successfully executed. It demonstrated insertion of an element in an array at the beginning, at any specified position, and at the end by shifting the required elements and displaying the updated array.

ASSIGNMENT:

- WAP to Insert Multiple Elements into an Array
- WAP to Insert an Element in a Sorted Array
- WAP to Insert an Element Only if Space is Available
- WAP to Insert an Element Before a Given Element
- WAP to Insert an Element After a Given Element

EXPERIMENT NO. 4

WRITE A PROGRAM IN C++ TO IMPLEMENT DELETION OF AN ELEMENT FROM AN ARRAY AT THE BEGINNING, ANY POSITION, AND THE END.

AIM: To study and implement deletion of an element from an array at the beginning, at any specified position, and at the end using C++

SOFTWARE USED: Visual Studio Code / Code: Blocks / Dev-C++, GCC/G++ Compiler. Operating System: Windows/Linux

THEORY: An array is a collection of elements stored in contiguous memory locations. Deletion is the process of removing an existing element from an array. Since arrays occupy consecutive memory locations, deleting an element creates an empty space. To maintain continuity, the remaining elements are shifted towards the left.

Deletion can be performed in three ways:

- **Deletion at the Beginning:** The first element is removed, and all remaining elements are shifted one position towards the left.
- **Deletion at Any Position:** The specified element is removed, and all subsequent elements are shifted one position towards the left.
- **Deletion at the End:** The last element is removed by simply reducing the size of the array. No shifting is required.

Deletion operation is commonly used in database management systems, inventory systems, employee management systems, and dynamic data processing.

ALOGRITHM:

1. Start.
2. Declare an integer array.
3. Enter the number of elements.
4. Read the array elements.
5. Display the insertion menu.
6. If beginning is selected, shift all elements one position towards the left.
7. If any position is selected, shift elements after the specified position towards the left.
8. If end is selected, decrease the array size by one
9. Display the updated array.
10. Stop

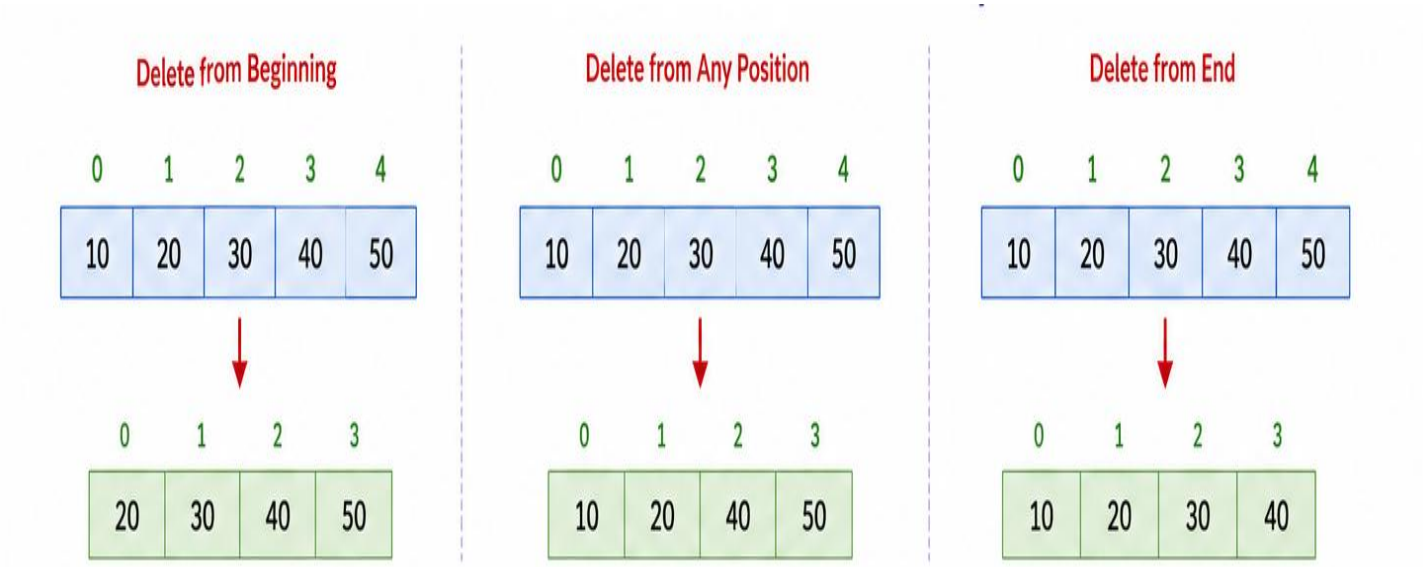


Figure.1: Deletion of an Element from an Array

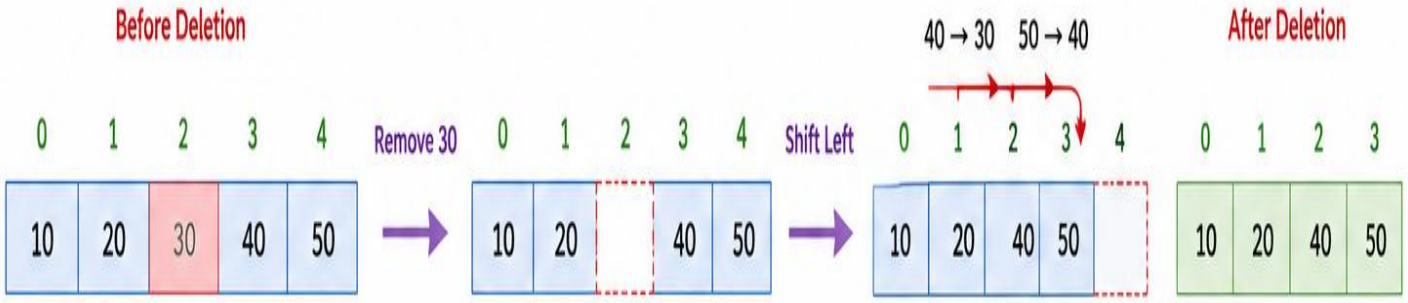


Figure.2: Element Shifting during Deletion (Ex: Delete 30 at position 3)

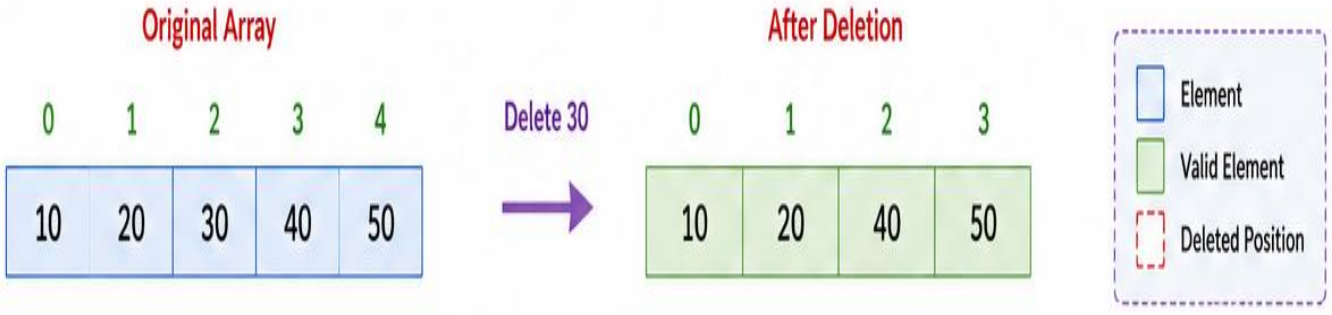


Figure.3: Memory Representation

PROGRAM (C++):

```
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, value, pos, choice;

    cout<<"Enter number of elements: ";
    cin>>n;

    cout<<"Enter array elements:\n";
    for(int i=0;i<n;i++)
        cin>>arr[i];

    cout<<"\n1. Delete at Beginning";
    cout<<"\n2. Delete at Any Position";
    cout<<"\n3. Delete at End";
    cout<<"\nEnter your choice: ";
    cin>>choice;

    if(choice==1)
    {
        cout<<"Enter value: ";
        cin>>value;

        for(int i=0;i<n-1;i++)
            arr[i]=arr[i+1];

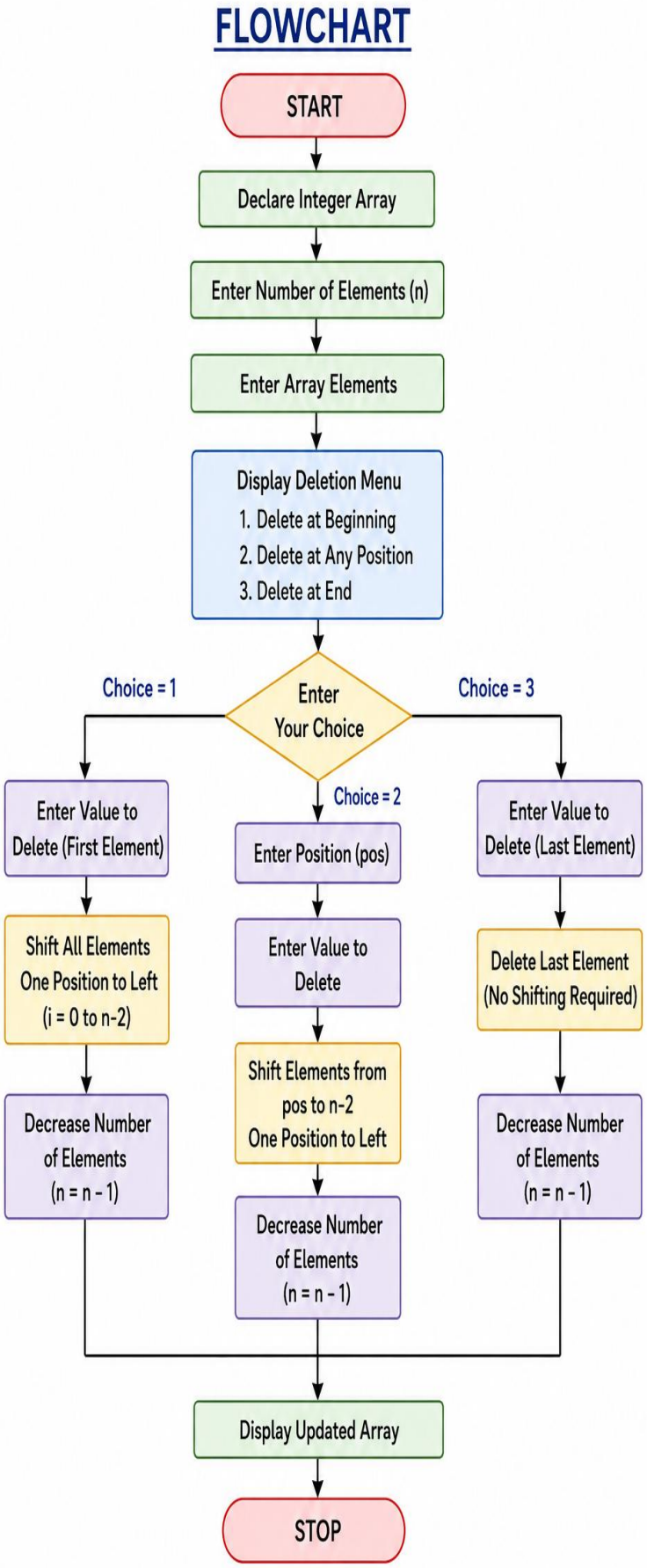
        n--;
    }
    else if(choice==2)
    {
        cout<<"Enter Position: ";
        cin>>pos;

        cout<<"Enter value: ";
        cin>>value;

        for(int i=pos-1;i<n-1;i++)
            arr[i]=arr[i+1];
        n--;
    }
    else if(choice==3)
    {
        cout<<"Enter value: ";
        cin>>value;
        n--;
    }

    cout<<"\nUpdated Array:\n";
    for(int i=0;i<n;i++)
        cout<<arr[i]<<" ";

    return 0;
}
```



EXPLANATION OF THE PROGRAM:

Statement	Explanation
#include <iostream>	Provides input/output functions.
int arr[100];	Declares an integer array with extra space.
cin>>n;	Reads the number of elements in the array.
for(int i=0;i<n;i++)	Reads the array elements.
choice	Selects the type of insertion.
for(int i=0;i<n-1;i++)	Shifts all elements left for deletion at the beginning.
for(int i=pos-1;i<n-1;i++)	Shifts elements left after deleting from the specified position.
n--;	Reduces the size of the array after deletion.
cout	Displays the updated array.

Sample Input 1 (Beginning):
Enter number of elements: 5
10 20 30 40 50
Choice: 1
Sample Output:
Updated Array:
20 30 40 50
Sample Input 2 (Any Position):
Enter number of elements: 5
10 20 30 40 50
Choice: 2
Enter Position: 3
Sample Output:
Updated Array:
10 20 40 50
Sample Input 3 (End):
Enter number of elements: 5
10 20 30 40 50
Choice: 3
Sample Output:
Updated Array:
10 20 30 40
Time Complexity

Delete at Beginning: **O(n)**
Delete at Any Position: **O(n)**
Delete at End: **O(1)**

Space Complexity

O (1)

IMPLEMENTATION OF CODE:

```cpp
#include <iostream>
using namespace std;

int main()
{
    int arr[100], n, pos, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter array elements:\n";
    for(int i=0; i<n; i++)
        cin >> arr[i];

    cout << "\n1. Delete from Beginning";
    cout << "\n2. Delete from Any Position";
    cout << "\n3. Delete from End";
    cout << "\nEnter your choice: ";
    cin >> choice;

    if(choice == 1)
    {
        // Shift all elements left
        for(int i=0; i<n-1; i++)
            arr[i] = arr[i+1];

        n--;
    }

    else if(choice == 2)
    {
        cout << "Enter Position: ";
        cin >> pos;

        for(int i=pos-1; i<n-1; i++)
            arr[i] = arr[i+1];

        n--;
    }

    else if(choice == 3)
    {
        // Delete last element
        n--;
    }

    cout << "\nUpdated Array:\n";

    for(int i=0; i<n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

```
Enter number of elements: 5
Enter array elements:
10 20 30 40 50

1. Delete from Beginning
2. Delete from Any Position
3. Delete from End
Enter your choice: 1

Updated Array:
20 30 40 50

...Program finished with exit code 0
Press ENTER to exit console.
```

APPLICATIONS: Student Record Management, Employee Database, Inventory Management, Hospital Management System, Database Management Systems.

RESULT: The program was successfully executed. It demonstrated deletion of an element from an array at the beginning, at any specified position, and at the end by shifting the remaining elements appropriately and displaying the updated array.

ASSIGNMENT:

- WAP to delete an element from the beginning of an array
- WAP to delete an element from a specified position in an array
- WAP to delete an element from the end of an array
- WAP to delete a given element by searching its value in an array
- Write a menu-driven C++ program to perform insertion and deletion operations on an array.